

DAY 4

Transformer Decoder • BERT vs GPT • Training • Inference • Interview Revision

Core idea: The encoder primarily builds contextual representations of an input sequence, while the decoder generates an output sequence autoregressively.

1. Encoder vs Decoder

Feature	Encoder	Decoder
Main purpose	Understand / represent input	Generate output sequence
Self-attention	Bidirectional	Masked / causal
Future tokens	Visible	Hidden
Cross-attention	No	Yes in the original encoder-decoder Transformer
Typical example	BERT	GPT-style decoder

Memory trick: Encoder = understand. Decoder = generate.

2. Transformer Decoder From Scratch

The decoder generates output tokens using the tokens generated so far and, in the original encoder-decoder Transformer, information from the encoder.

Decoder input
↓
Masked Multi-Head Self-Attention
↓
Residual + LayerNorm
↓
Multi-Head Cross-Attention ← Encoder output
↓
Residual + LayerNorm
↓
Feed-Forward Network
↓
Residual + LayerNorm
↓
Decoder output

Important: Tokenization, embeddings, and positional information occur before the decoder block stack. Multiple decoder blocks are then stacked.

2.1 Tokenization and Embeddings

Text is split into tokens. Each token receives an ID and is mapped to a learned vector embedding. Positional information is also incorporated so the model knows token order.

Text
↓
Tokens

↓
Token IDs
↓
Embeddings + positional information
↓
Decoder blocks

2.2 Masked Self-Attention

The decoder's self-attention is causal. Each position can attend to itself and earlier positions, but not future positions.

The cat sat today
The ✓ ✗ ✗ ✗
cat ✓ ✓ ✗ ✗
sat ✓ ✓ ✓ ✗
today ✓ ✓ ✓ ✓

The upper-triangular future region is masked, typically by adding negative infinity before softmax. Those future positions therefore receive effectively zero attention probability.

Why? Future tokens would leak the answer during next-token prediction. The model must learn to predict the future from the past.

2.3 Multi-Head Self-Attention

For self-attention, queries, keys, and values all originate from the decoder's current representations:

$Q = X_{\text{decoder}} W_Q$
 $K = X_{\text{decoder}} W_K$
 $V = X_{\text{decoder}} W_V$

 $\text{Attention}(Q, K, V)$
 $= \text{softmax}(QK^T / \sqrt{d_k}) V$

Multiple heads allow the model to learn different relationships between tokens.

2.4 Cross-Attention

Cross-attention allows the decoder to use information from the encoder. The decoder queries the encoder representations.

$Q = \text{decoder representation}$
 $K = \text{encoder output}$
 $V = \text{encoder output}$

Mental model: masked self-attention asks, What have I generated so far? Cross-attention asks, What information from the input do I need?

The reason Q comes from the decoder is that the decoder determines what information it currently needs. The encoder supplies keys for matching and values containing the information to retrieve.

2.5 Feed-Forward Network

After attention, each position is processed by a feed-forward neural network, usually an MLP with a nonlinear activation.

Linear
↓

Activation
↓
Linear

Useful intuition: Attention lets tokens exchange information; the FFN transforms the resulting representation.

3. Full Decoder Block

Input embeddings + positional information
↓
Masked Multi-Head Attention
↓
Residual + LayerNorm
↓
Cross-Attention
↑ |
| ↓
Encoder output
↓
Residual + LayerNorm
↓
FFN
↓
Residual + LayerNorm
↓
Block output

This block is repeated many times. After the final decoder block, the hidden states are projected onto the vocabulary.

4. From Hidden State to Next Token

Final decoder hidden states
↓
Linear projection
↓
Vocabulary logits
↓
Softmax / probabilities
↓
Next token

During training, the logits are compared with the correct target tokens using a loss such as cross-entropy. In practice, cross-entropy is commonly computed directly from the logits without explicitly materializing softmax probabilities.

5. Training vs Inference

Stage	What happens?
Training	The whole target sequence can be processed in parallel while causal masking prevents future-token
Inference	The model generates autoregressively, selecting one token and feeding it back for the next prediction

Important interview distinction: Training can be parallel; inference is sequential/autoregressive.

6. Teacher Forcing

Teacher forcing means feeding the decoder the ground-truth previous tokens during training instead of its own previous predictions.

Decoder input: <BOS> I love cats
Target: I love cats <EOS>

This makes training more stable and efficient and lets the model learn next-token predictions from the correct previous context.

7. GPT From the Decoder Perspective

GPT is a decoder-style Transformer trained autoregressively for next-token prediction. Unlike the original encoder-decoder Transformer, GPT does not have a separate encoder.

GPT:
Token embeddings
↓
Causal / masked self-attention
↓
FFN
↓
Causal / masked self-attention
↓
FFN
↓
...
↓
Next-token logits

Critical: GPT uses causal/masked self-attention. What GPT removes is cross-attention, because there is no separate encoder output.

8. Why GPT Has No Cross-Attention

The original Transformer has separate source and target sequences. The decoder therefore needs cross-attention to retrieve information from the encoded source.

Original Transformer:

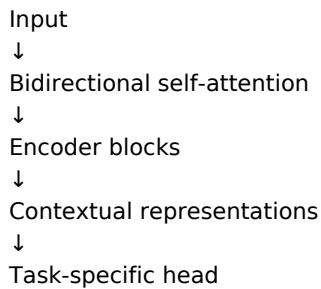
Source → Encoder → encoder representations
↓
K and V
↑
Decoder Q

GPT instead operates on a single autoregressive sequence. Its previous tokens are already its context, so causal self-attention provides the required context.

Key distinction: Original decoder = previous output context + encoder context. GPT = previous token context only.

9. BERT

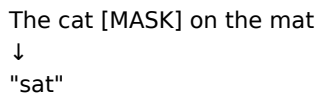
BERT stands for Bidirectional Encoder Representations from Transformers. BERT uses the Transformer encoder architecture.



BERT's self-attention is bidirectional: a token can attend to tokens on both sides.

BERT Training: Masked Language Modeling

BERT is commonly associated with masked language modeling. Some tokens are hidden and the model predicts them using the surrounding context.



Because the model can use both left and right context, this objective is fundamentally different from GPT's causal next-token prediction.

10. BERT vs GPT

Feature	BERT	GPT
Architecture	Encoder	Decoder-style
Attention	Bidirectional self-attention	Causal / masked self-attention
Cross-attention	No	No
Training objective	Masked language modeling	Autoregressive next-token prediction
Future tokens during attention	Visible	Hidden
Typical strength	Language understanding / representation	Text generation

Core memory: BERT sees both directions to build contextual representations. GPT sees only previous tokens so it can generate the next token.

11. Why BERT Fits Classification

Sentiment classification requires understanding the input as a whole. BERT's bidirectional attention lets each token use both preceding and following context. A representation such as the [CLS] token can then be passed to a classification head.

12. Why GPT Fits Generation

Generation requires predicting future tokens without seeing them. GPT's causal attention enforces this setup, and its training objective directly teaches it to predict the next token.

13. High-Yield Interview Questions

- Q1. What is the purpose of the Transformer decoder?
- Q2. Why does the decoder use masked/causal self-attention?
- Q3. What is cross-attention and where do Q, K, and V come from?
- Q4. Walk through one complete decoder block.
- Q5. Why does the decoder need both self-attention and cross-attention?
- Q6. Why does Q come from the decoder while K and V come from the encoder?
- Q7. How can training process the whole target sequence without leaking future tokens?
- Q8. What is teacher forcing?
- Q9. How do decoder hidden states become token predictions?
- Q10. Why does GPT not need cross-attention?
- Q11. What is the fundamental architectural difference between BERT and GPT?
- Q12. Why doesn't BERT use causal masking?
- Q13. What is BERT's masked language modeling objective?
- Q14. Why is BERT suited to classification while GPT is suited to generation?

14. Interview-Ready Summary

Transformer Decoder: The decoder generates an output sequence autoregressively. Each block contains masked multi-head self-attention, cross-attention to encoder outputs in the original encoder-decoder architecture, and a feed-forward network, with residual connections and normalization. Masked self-attention prevents future-token leakage, while cross-attention lets the decoder retrieve relevant source information.

BERT: Encoder-only, bidirectional self-attention, masked language modeling, strong for contextual representations and understanding tasks.

GPT: Decoder-style, causal self-attention, autoregressive next-token prediction, strong for generation. GPT has no cross-attention because it has no separate encoder.

15. Final Mental Model

ENCODER

Understand the input

↓

Bidirectional self-attention

ORIGINAL DECODER

1. Look at previous output tokens

↓

2. Look at relevant encoder information

↓

3. Process with FFN

↓

4. Predict next token

BERT

Encoder-only

Bidirectional

Masked-token prediction

GPT

Decoder-style

Causal

Next-token prediction

One-line memory trick: Encoder = understand. Decoder = generate. BERT = bidirectional understanding.
GPT = causal generation.